

CASE STUDY: AI/NLP RISK ENGINE

AI/NLP Risk Engine

From unstructured news and tweets to portfolio actions: sentiment, event class and impact scores that drive an index rebalancer and a bank stress test.

Durgesh Bhatt

IIIT Bhubaneswar | S&P Global & Crisil Campus Hackathon 2026

Problem and approach

The problem. Risk signals sit in news and social posts, and they arrive faster than analysts can read. The case study asks for one engine that turns text into structured signals, plus a downstream module that proves they are useful.



1 Ingest

News and tweets are cleaned, mapped to one schema, linked to tickers (\$AAPL, names) and de-duplicated.



2 Score

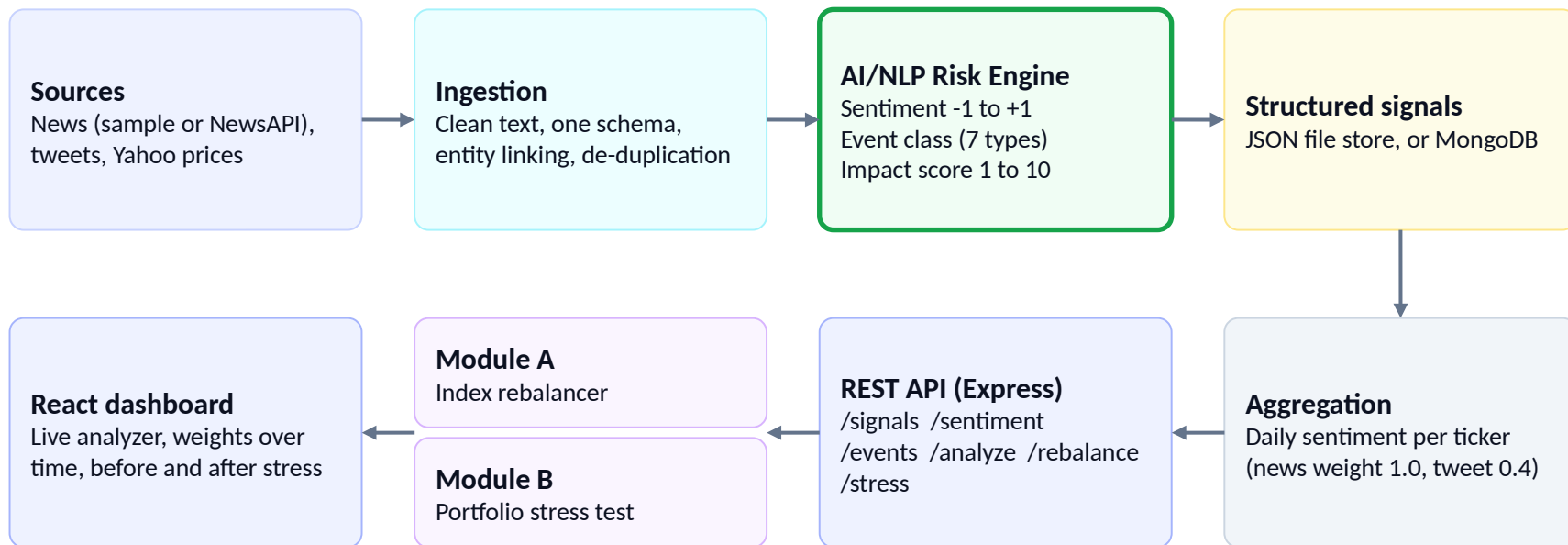
Every document gets a sentiment score (-1 to +1), an event class (7 types) and an impact score (1 to 10), with the matched terms as evidence.



3 Act

Module A rebalances a 15-stock index. Module B stress-tests a bank book when an adverse event scores above 7. I built both.

System design



One contract. Every document becomes a Signal; both modules read only that.

Zero setup. JSON file store by default, MongoDB optional behind one interface.

Explainable. Every score lists the terms that drove it.

Implementation highlights

Ingestion

Entity linking from cashtags and company names, 100% precision and recall on the sample. Syndicated duplicates removed with n-gram similarity: 4 of 4 caught, no false removals.

NLP engine

Finance lexicon with negation and keyword event rules, plus optional FinBERT and zero-shot models (transformers.js, runs locally). Impact = event severity adjusted by sentiment strength.

Module A: rebalancer

Sentiment memory, tilt, 2-15% weight caps, 10% daily turnover limit, 5 bp costs. Weights set at close earn the next day's return, so there is no look-ahead.

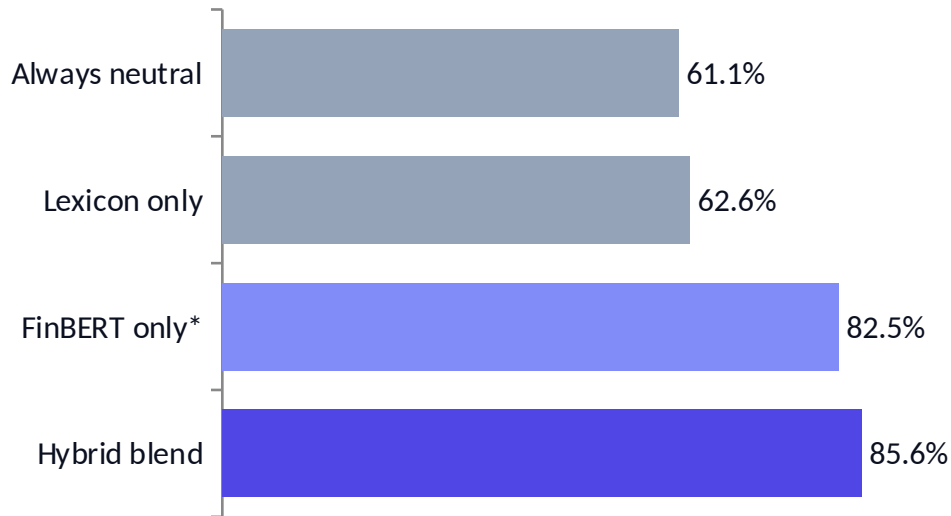
Module B: stress test

Triggers on impact above 7 and adverse sentiment. Shocks scale with impact; bonds, loans, equities and swaps are revalued with duration, beta and DV01. Company events spill over 30% to the sector.

TypeScript | Node + Express | Zod | React + Recharts | 46 automated tests

Key results

Sentiment accuracy, 1,000 Financial PhraseBank sentences



*FinBERT was trained on PhraseBank, so its bar is optimistic. The lexicon never saw this data, so its bar is a true out-of-domain score.

91.9%

event-type accuracy, synthetic sample, with zero-shot fallback (rules only: 89%)

-5.21%

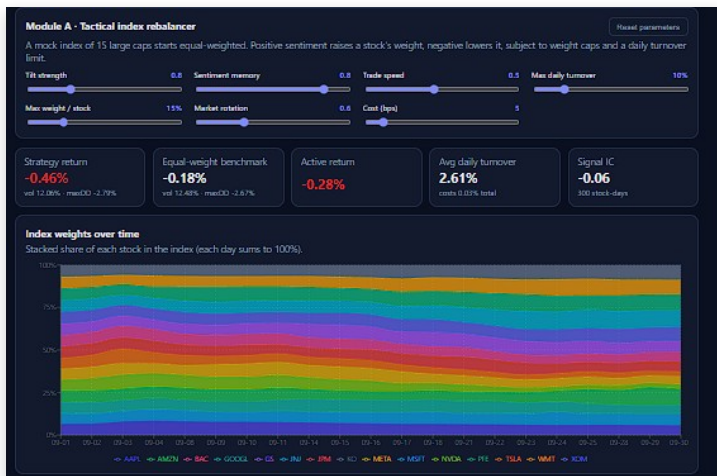
synthetic bank book after a Geopolitical impact-9 shock: \$1,644.5m to \$1,558.8m

29 of 36

high-impact signals are adverse and trigger a stress test

Module A: -0.46% vs -0.18% equal weight, IC -0.06. News is synthetic, so this validates the machinery, not alpha.

Domain impact: headline to portfolio action



Module A: weights shift with sentiment, within caps



Module B: before and after a Geopolitical impact-9 event



Speed

Analysts read dozens of headlines a day; the engine scores thousands and surfaces only severe ones.



Traceability

Each signal lists the terms behind it, so a decision can be explained and audited.



Actionability

One signal drives a tactical decision (rebalancing) and a strategic one (stress testing).

Limitations and next steps

What I would not claim

- Text is synthetic; accuracy on real news is only as good as the PhraseBank test shows.
- Lexicon is weak on company-results wording; FinBERT is weak on macro headlines and tweets.
- Backtest covers about one month, so it is a pipeline check, not evidence of alpha.
- Stress model is simplified: no convexity, netting or collateral; shock sizes are assumptions.

Next steps

- Route sentiment weights by source and scope, and expand the lexicon.
- Evaluate on a fresh, real labelled news set.
- Stream live news instead of batch plus on-demand.
- Calibrate shocks to historical episodes and add multi-event scenarios.

